

Basic Data Structures: Stacks and Queues

Neil Rhodes

Department of Computer Science and
Engineering University of California, San Diego

Data Structures
Data Structures and Algorithms

Outline

1 Stacks

2 Queues



LIFO = Last In - First Out

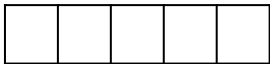
Definition

Stack: Abstract data type with the following operations:

- `Push(key)`: adds key to collection
- `key Top()`: returns most recently-added key
- `key Pop()`: removes and returns most recently-added key
- `Boolean Empty()`: are there any elements?

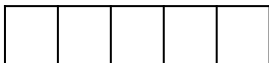
Stack Implementation with Array

numElements: 0



Stack Implementation with Array

numElements: 0



Push (a)

Stack Implementation with Array

numElements: 1



Push (a)

Stack Implementation with Array

numElements: 1



Stack Implementation with Array

numElements: 1



Push (b)

Stack Implementation with Array

numElements: 2



Push (b)

Stack Implementation with Array

numElements: 2



Stack Implementation with Array

numElements: 2



Top ()

Stack Implementation with Array

numElements: 2



Top () $\rightarrow$ b

Stack Implementation with Array

numElements: 2



Stack Implementation with Array

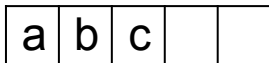
numElements: 2



Push (c)

Stack Implementation with Array

numElements: 3



Push (c)

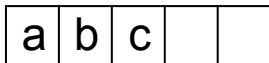
Stack Implementation with Array

numElements: 3

a	b	c		
---	---	---	--	--

Stack Implementation with Array

numElements: 3



Pop ()

Stack Implementation with Array

numElements: 2



Pop () $\rightarrow$ c

Stack Implementation with Array

numElements: 2



Stack Implementation with Array

numElements: 2



Push (d)

Stack Implementation with Array

numElements: 3

a	b	d		
---	---	---	--	--

Push (d)

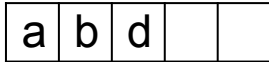
Stack Implementation with Array

numElements: 3

a	b	d		
---	---	---	--	--

Stack Implementation with Array

numElements: 3



Push (e)

Stack Implementation with Array

numElements: 4

a	b	d	e	
---	---	---	---	--

Push (e)

Stack Implementation with Array

numElements: 4

a	b	d	e	
---	---	---	---	--

Stack Implementation with Array

numElements: 4

a	b	d	e	
---	---	---	---	--

Push (f)

Stack Implementation with Array

numElements: 5

a	b	d	e	f
---	---	---	---	---

Push (f)

Stack Implementation with Array

numElements: 5

a	b	d	e	f
---	---	---	---	---

Stack Implementation with Array

numElements: 5

a	b	d	e	f
---	---	---	---	---

Push (g)

Stack Implementation with Array

numElements: 5

a	b	d	e	f
---	---	---	---	---

Push (g) $\rightarrow$ ERROR

Stack Implementation with Array

numElements: 5

a	b	d	e	f
---	---	---	---	---

Stack Implementation with Array

numElements: 5

a	b	d	e	f
---	---	---	---	---

Empty ()

Stack Implementation with Array

numElements: 5

a	b	d	e	f
---	---	---	---	---

Empty() $\rightarrow$ False

Stack Implementation with Array

numElements: 5

a	b	d	e	f
---	---	---	---	---

Stack Implementation with Array

numElements: 5

a	b	d	e	f
---	---	---	---	---

Pop()

Stack Implementation with Array

numElements: 4

a	b	d	e	
---	---	---	---	--

Pop () $\rightarrow$ f

Stack Implementation with Array

numElements: 4

a	b	d	e	
---	---	---	---	--

Stack Implementation with Array

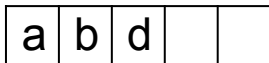
numElements: 4

a	b	d	e	
---	---	---	---	--

Pop()

Stack Implementation with Array

numElements: 3



Pop () $\rightarrow$ e

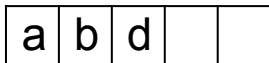
Stack Implementation with Array

numElements: 3

a	b	d		
---	---	---	--	--

Stack Implementation with Array

numElements: 3



Pop()

Stack Implementation with Array

numElements: 2



Pop () $\rightarrow$ d

Stack Implementation with Array

numElements: 2



Stack Implementation with Array

numElements: 2



Pop()

Stack Implementation with Array

numElements: 1



Pop () $\rightarrow$ b

Stack Implementation with Array

numElements: 1



Stack Implementation with Array

numElements: 1



Pop()

Stack Implementation with Array

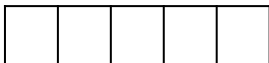
numElements: 0



Pop () $\rightarrow$ a

Stack Implementation with Array

numElements: 0



Stack Implementation with Array

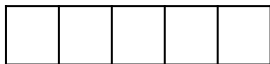
numElements: 0



Empty()

Stack Implementation with Array

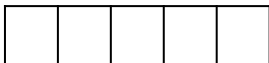
numElements: 0



Empty () $\rightarrow$ True

Stack Implementation with Array

numElements: 0



<https://www.onlinegdb.com>

Outline

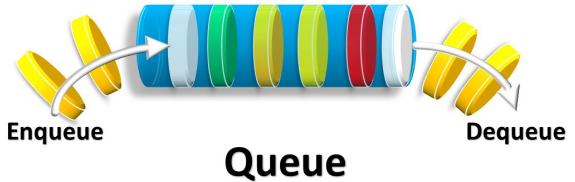
1 Stacks



FIFO (First in First Out)



2 Queues



Colas

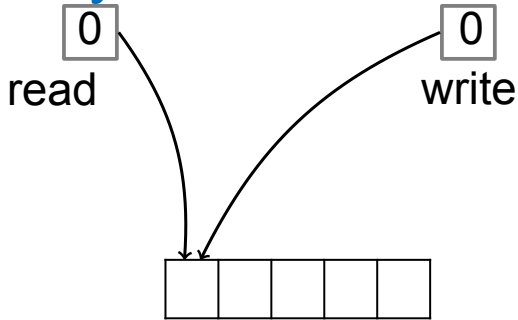
Definition

Queue: Abstract data type with the following operations:

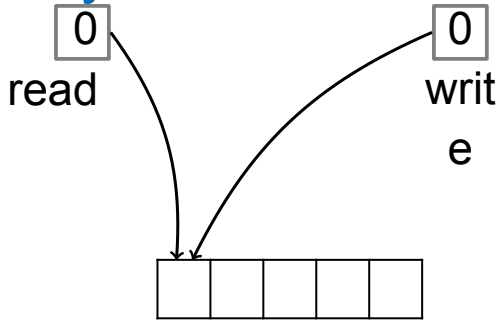
- `Enqueue(key)`: adds key to collection
- `key Dequeue()`: removes and returns least recently-added key
- `Boolean Empty()`: are there any elements?

FIFO: First-In, First-Out

Queue Implementation with Array

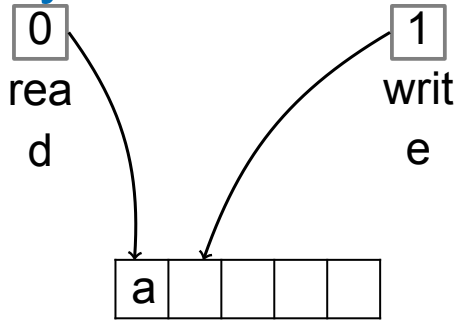


Queue Implementation with Array



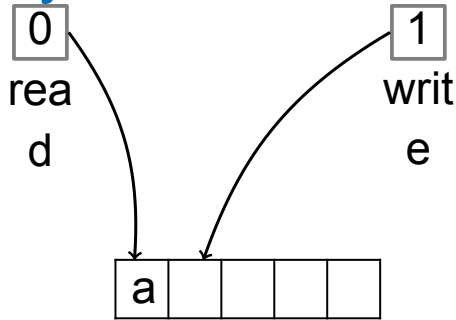
Enqueue(a)

Queue Implementation with Array

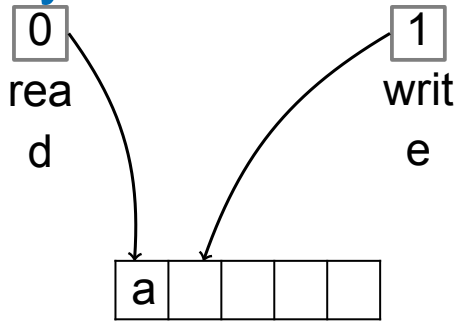


Enqueue(a)

Queue Implementation with Array

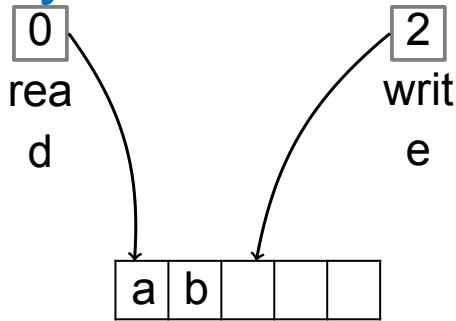


Queue Implementation with Array



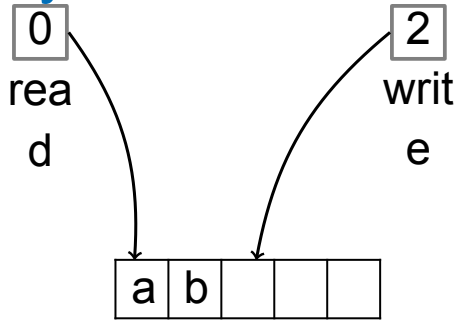
Enqueue (b)

Queue Implementation with Array

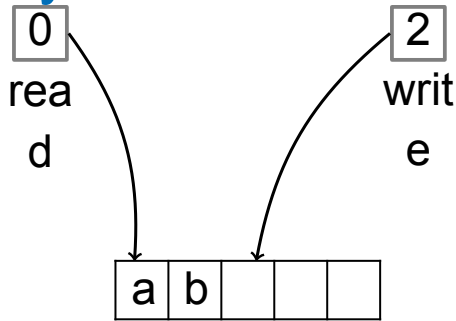


Enqueue (b)

Queue Implementation with Array

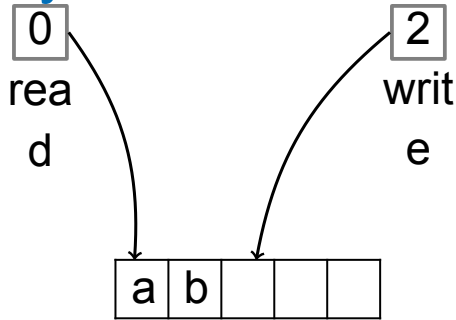


Queue Implementation with Array



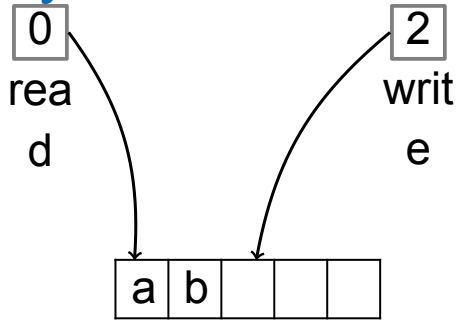
Empty ()

Queue Implementation with Array

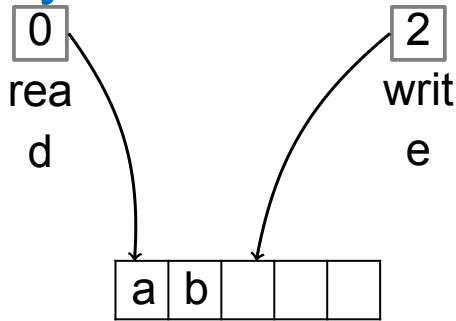


`Empty ()` $\rightarrow$ `False`

Queue Implementation with Array

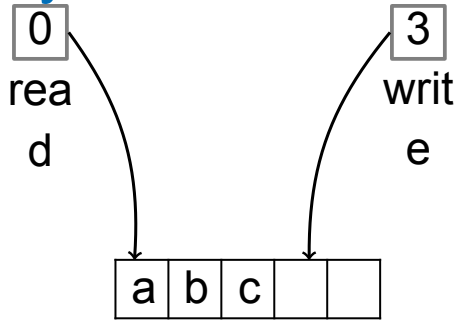


Queue Implementation with Array



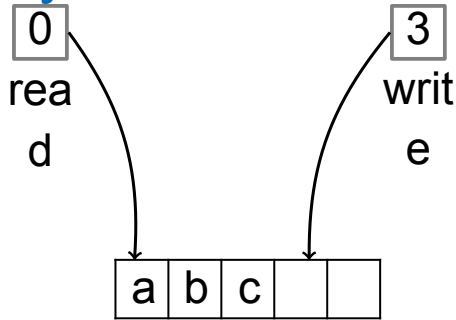
Enqueue (c)

Queue Implementation with Array

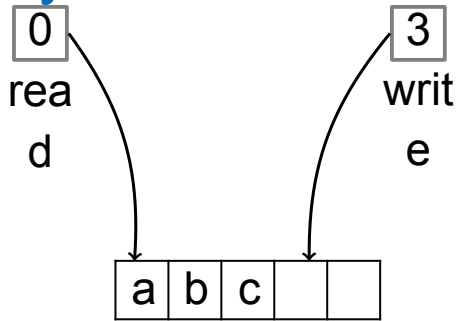


Enqueue (c)

Queue Implementation with Array

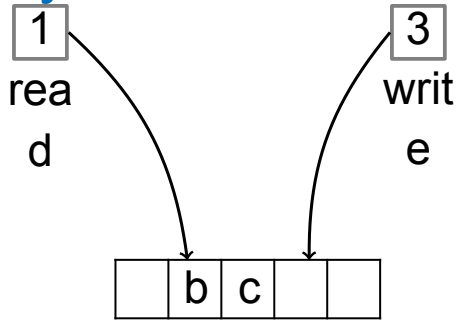


Queue Implementation with Array



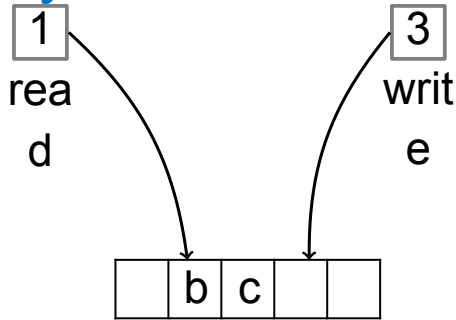
Dequeue ()

Queue Implementation with Array

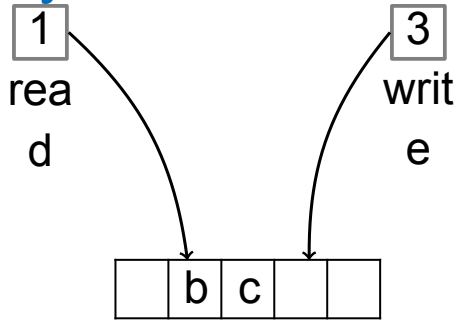


Dequeue () $\rightarrow$ a

Queue Implementation with Array

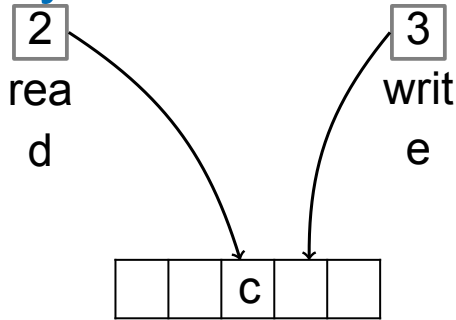


Queue Implementation with Array



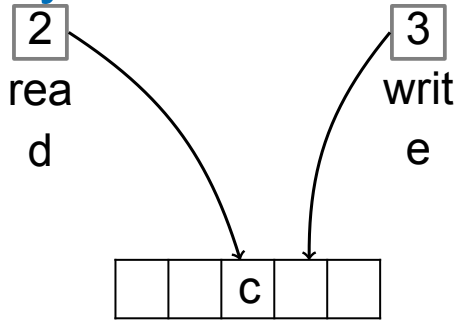
Dequeue ()

Queue Implementation with Array

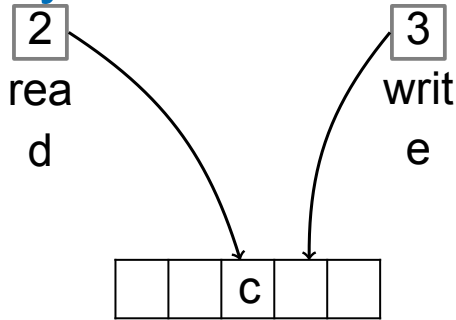


Dequeue () $\rightarrow$ b

Queue Implementation with Array

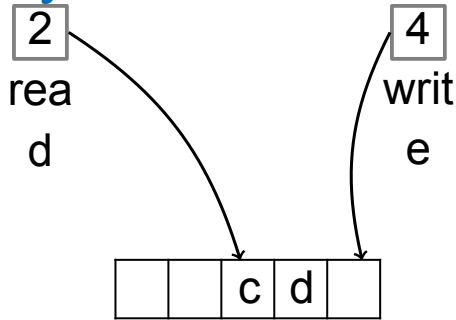


Queue Implementation with Array



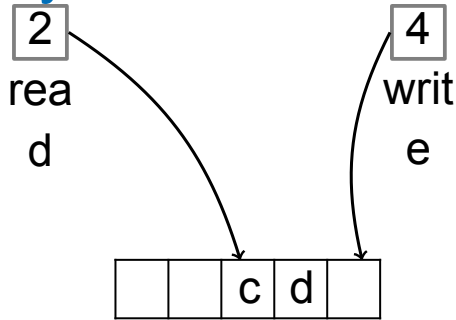
Enqueue (d)

Queue Implementation with Array

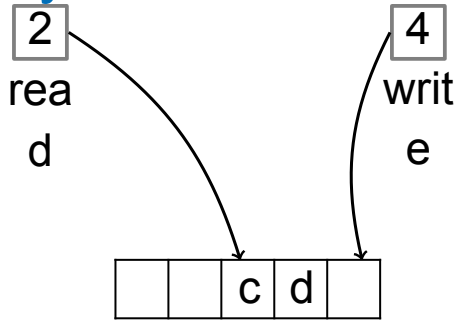


Enqueue (d)

Queue Implementation with Array

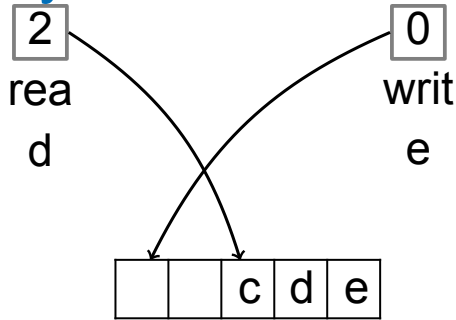


Queue Implementation with Array



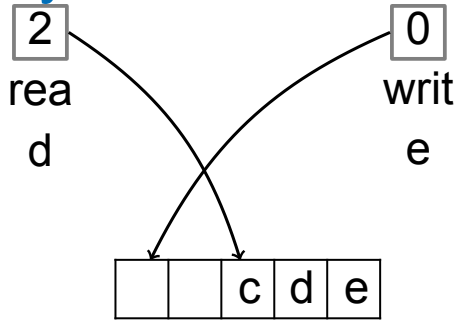
Enqueue (e)

Queue Implementation with Array

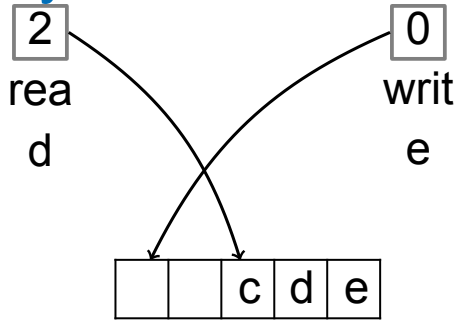


Enqueue (e)

Queue Implementation with Array

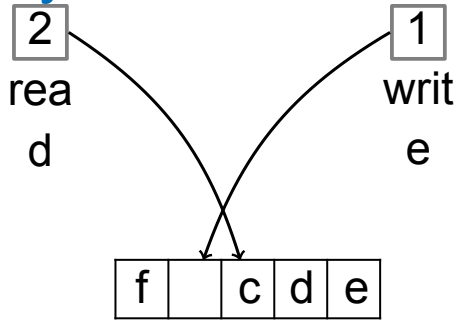


Queue Implementation with Array



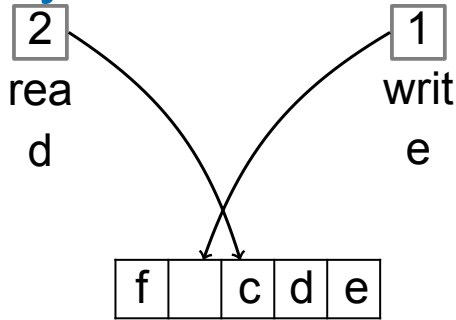
Enqueue (f)

Queue Implementation with Array

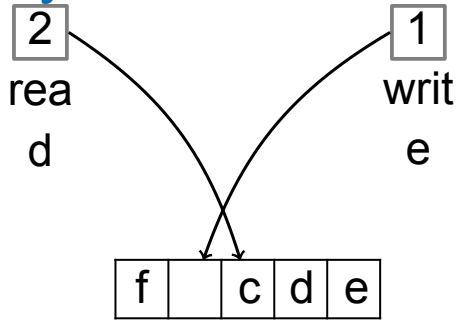


Enqueue (f)

Queue Implementation with Array

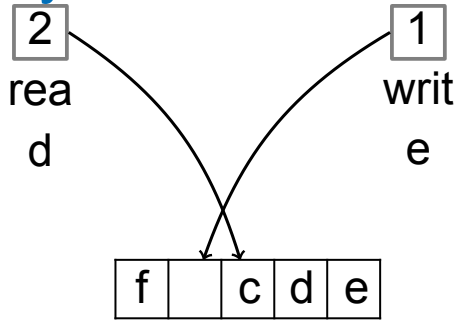


Queue Implementation with Array



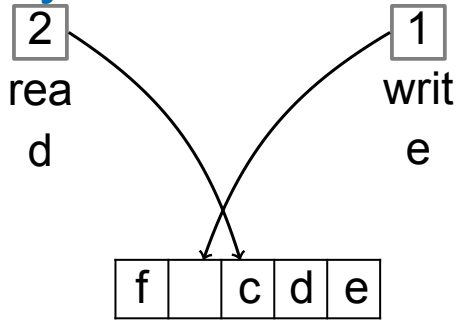
Enqueue (g)

Queue Implementation with Array

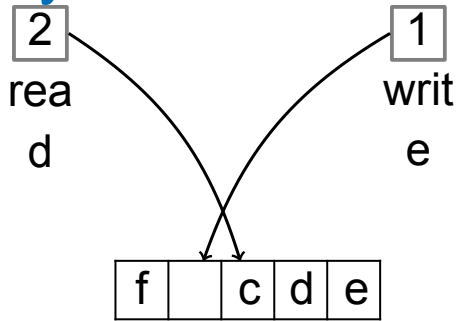


Enqueue (g) $\rightarrow$ ERROR

Queue Implementation with Array

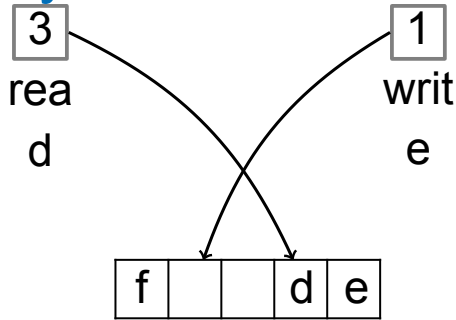


Queue Implementation with Array



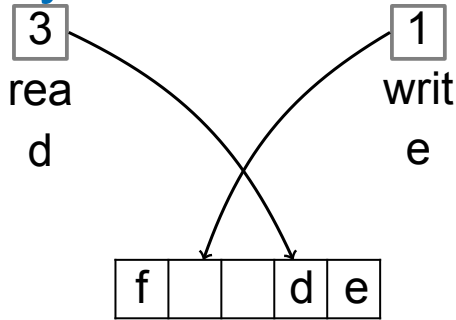
Dequeue ()

Queue Implementation with Array

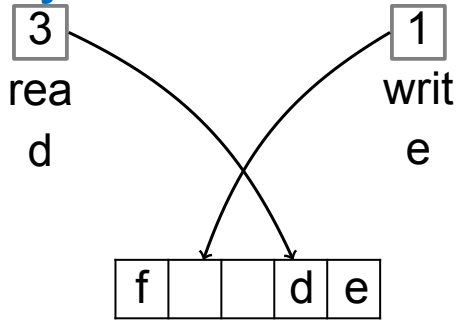


Dequeue () $\rightarrow$ c

Queue Implementation with Array

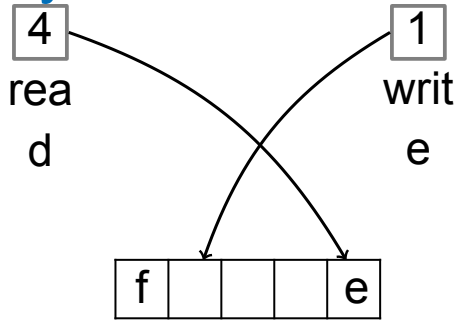


Queue Implementation with Array



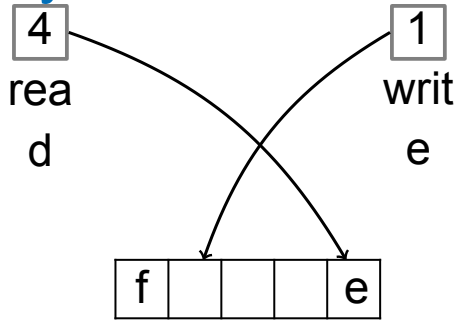
Dequeue ()

Queue Implementation with Array

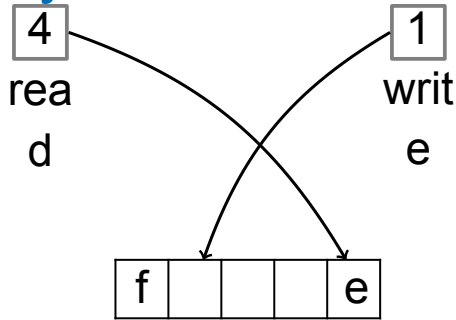


Dequeue () $\rightarrow$ d

Queue Implementation with Array

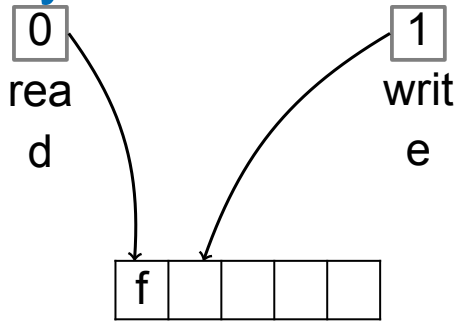


Queue Implementation with Array



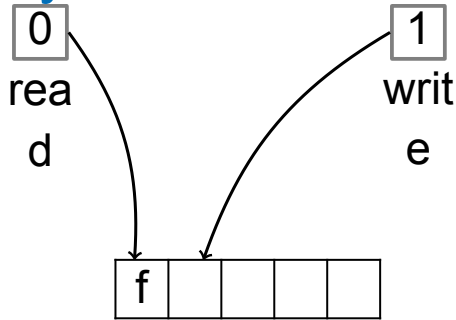
Dequeue ()

Queue Implementation with Array

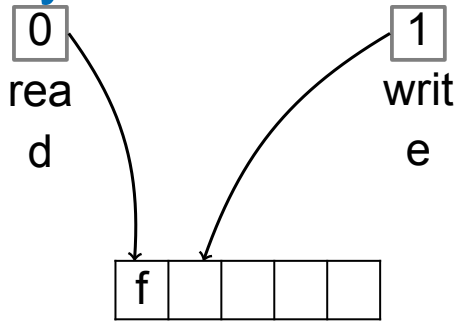


Dequeue () $\rightarrow$ e

Queue Implementation with Array

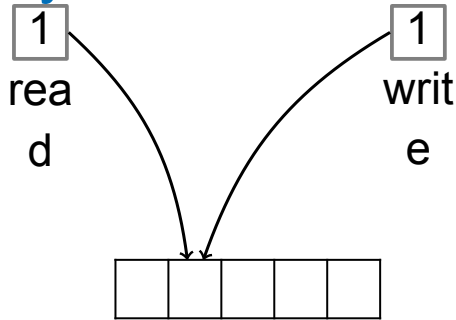


Queue Implementation with Array



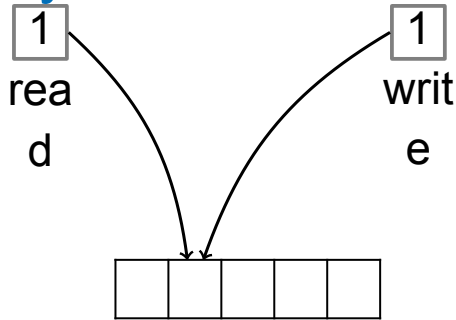
Dequeue ()

Queue Implementation with Array

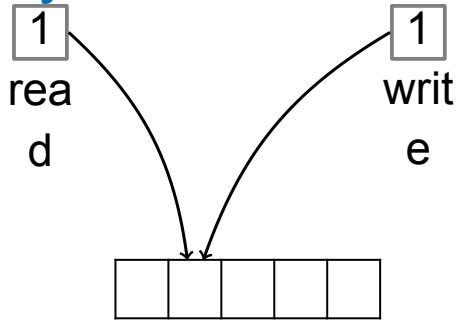


Dequeue () $\rightarrow$ f

Queue Implementation with Array

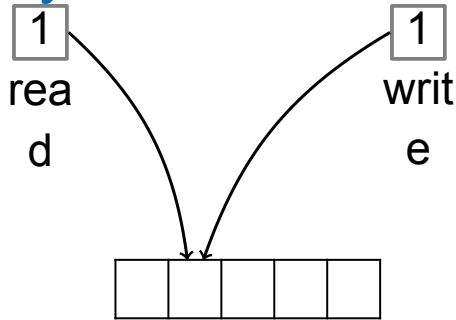


Queue Implementation with Array



Empty ()

Queue Implementation with Array



`Empty ()` $\rightarrow$ `True`

Summary

- Queues can be implemented with either a linked list (with tail pointer) or an array.
- Each queue operation is $O(1)$: Enqueue, Dequeue, Empty.

<https://sites.google.com/unal.edu.co/data/>